

Sorting

Sorting ని ఎవరూ మళ్ళీ రాయరు. `.sort()` అని రాసి ముందుకి వెళ్తారు. మరియు అక్కడే మూడు bugs దాక్కుంటాయి – ఒక్కటే crash కాకుండా.

మొదటిది – `.sort()` comparator లేకుండా, మరియు మీ test data సరిగ్గా 0.0% విఫలం చేస్తుంది. రెండోది – `? 1 : -1` అనే comparator 21,000 runs lo ఒక్క array నీ తప్పుగా sort చేయలేదు, మరియు స్థిరత్వాన్ని 100% నాశనం చేసింది. మూడోది – median-of-three quicksort ని sorted input నుంచి కాపాడుతుంది, మరియు నకళ్ళ దగ్గర అది 2,001x నెమ్మది.

ఈ doc ఎలా చదవాలి

పక్కన editor తెరిచి కలిసి రాయండి. ఇక్కడి ప్రతి output, ప్రతి సంఖ్యా నిజంగా `node` lo run చేసినదే – §10 యొక్క $2,001 \times$ ని నేను వెతుకుతూ కనుగొనలేదు; నా స్వంత fuzz కి పని-పరిమితి కలిపినప్పుడు నా code lo అది బయటపడింది.

SORTING ఈ SERIES LO ప్రత్యేకం: ALGORITHM మీరు రాయరు

Deep 01-08 lo bug మీ algorithm lo ఉంటుంది. ఇక్కడ algorithm **V8** రాసింది, మరియు అది సరైనది.

మీరు రాసేది **ఒకే ఒక్క function** – comparator. మరియు మూడు విరుపుల్లో **రెండు** అందులోనే ఉన్నాయి.

కాబట్టి ఇక్కడ ప్రశ్న "ఏ sorting algorithm" కాదు – అది interview lo అడిగే ఆఖరి ప్రశ్న. ప్రశ్న: **"మీ comparator ఒక చెల్లుబాటు అయ్యే క్రమాన్ని నిర్వచిస్తోందా?"**

విషయ సూచిక (Table of Contents)

Part 1 – సమస్యని నిజంగా అర్థం చేసుకోవడం

1. Comparator ఒక ఒప్పందం
2. అడగాల్సిన ప్రశ్నలు

Part 2 – మొదటి విరుపు: comparator లేకపోవడం

3. Step – `.sort()` అని రాయడం
4. మొదటి విరుపు – 0.0% తప్పు, ఆపై 63.3%
5. Step – `a > b` ఎందుకు ఏమీ చేయదు

Part 3 – రెండో విరుపు: స్థిరత్వం

6. Step – రెండు keys ప్రకారం sort
7. రెండో విరుపు – 21,000 runs, 0 తప్పులు, 100% నష్టం
8. Step – `byKeys`

Part 4 – మూడో విరుపు: partition

9. Step – quicksort మరియు pivot
10. మూడో విరుపు – median-of-three నకళ్ళని చూడదు

Part 5 – నిరూపణ మరియు ప్రదర్శన

11. మొత్తం code
12. 24,000 cases · mutation testing
13. $n \log n$ ని దాటడం

Part 1 – సమస్యని నిజంగా అర్థం చేసుకోవడం

1. Comparator ఒక ఒప్పందం

`arr.sort(cmp)` అంటే "cmp ప్రకారం sort చెయ్యి" కాదు. అది ఒక **ఒప్పందం**:

COMPARATOR తప్పక పాటించాల్సినవి

1. `cmp(a, b) < 0` అయితే `a` ముందు `> 0` అయితే `b` ముందు `= 0` అయితే **సమానం**
2. **వ్యతిరేకత** – `cmp(a, b)` మరియు `cmp(b, a)` వ్యతిరేక గుర్తులు కలిగి ఉండాలి
3. **సంక్రమణ** – `a < b` మరియు `b < c` అయితే `a < c`
4. **సమానత్వం నిజమైనది** – `cmp(a, b) = 0` అయితే `a, b` ఏ క్రమంలోనైనా సరే

మీరు ఈ ఒప్పందాన్ని ఉల్లంఘిస్తే – spec ప్రకారం ఫలితం **undefined**. Engine crash కాదు, హెచ్చరించదు, ఒక "సరైనదిలా కనిపించే" జవాబు ఇస్తుంది.

§7 lo చూస్తూ: ఒప్పందం ఉల్లంఘించినా 21,000 runs lo ఒక్క array కూడా "sort కాలేదు" అనిపించలేదు.

2. అడగాల్సిన ప్రశ్నలు

ప్రశ్న	ఎందుకు అడుగుతున్నాం
స్థిరత్వం కావాలా?	§7 – రెండు keys ప్రకారం sort చేస్తుంటే తప్పక కావాలి
సమాన మూలకాలు ఉంటాయా?	ఉంటే §10 – partition ఎంపిక $O(n \log n)$ మరియు $O(n^2)$ మధ్య తేడా
ఏ రకం విలువలు – సంఖ్యలా, strings ఆ?	§4 – <code>.sort()</code> అన్నిటినీ string గా చూస్తుంది
Keys ఒక చిన్న పూర్ణాంక పరిధిలో ఉన్నాయా?	§13 – అయితే $n \log n$ పరిమితిని దాటొచ్చు
Data memory lo పడుతుందా?	పడకపోతే ఇది సమస్యే వేరు – §14
పూర్తి sort నిజంగా కావాలా?	Top-K కావాలంటే Deep 06 – <code>sort</code> 289 ms, heap 4.1 ms
Data ఇప్పటికే దాదాపు sorted ఆ?	అయితే TimSort $O(n)$ – కానీ మీ చేతి quicksort $O(n^2)$

ఒక ప్రశ్న అడిగితే మరు దీన్ని నిజంగా నడిపారని తెలుస్తుంది

"ఈ sort స్థిరమైనదా, మరియు మకు స్థిరత్వం అవసరమా?"

ఎందుకంటే జవాబు ప్రతిచోటా వేరు: JavaScript `Array.sort` ES2019 నుంచి స్థిరం · Java `Arrays.sort` objects కి స్థిరం, **primitives** కి కాదు · C++ `std::sort` స్థిరం కాదు, `std::stable_sort` వేరే function · Python `sorted` స్థిరం.

మరియు §7 lo చూస్తాం – స్థిరమైన sort వాడినా మీ comparator తప్పితే స్థిరత్వం పోతుంది.

Part 2 – మొదటి విరుపు: comparator లేకపోవడం

3. Step – `.sort()` అని రాయడం

```
const scores = [3, 1, 4, 1, 5, 9, 2, 6];
console.log(scores.sort()); // [1, 1, 2, 3, 4, 5, 6, 9] ✓
```

సరైనదే. ఒక పరీక్ష రాద్దాం:

```
const a = [7, 2, 9, 4, 1];
console.log(a.sort()); // [1, 2, 4, 7, 9] ✓
```

అదే సరైనదే. ఇంకో అయిదు ఉదాహరణలు రాసినా అన్నీ నెగ్గుతాయి.

4. మొదటి విరుపు – 0.0% తప్పు, ఆపై 63.3%

default `.sort()` – comparator లేకుండా

విలువల పరిధి	5,000 arrays లో తప్పు	%
0-9	0	0.0%
0-99	2,948	59.0%
0-999	3,166	63.3%
0-10 లక్షలు	3,237	64.7%

ఒక అంకె విలువలు – సరిగ్గా సున్నా వైఫల్యాలు

`.sort()` ప్రతి మూలకాన్నీ **string** గా మార్చి నిఘంటువు క్రమంలో పేరుస్తుంది.

ఒక అంకె సంఖ్యలకి నిఘంటువు క్రమం = సంఖ్యా క్రమం. "3" < "7" మరియు 3 < 7 – ఒకటే.

కాబట్టి మే test data ఈ bug ని చూపించలేదు. §3 లో నేను ఏ ఉదాహరణ రాసినా నెగ్గేది. రెండంకెల సంఖ్య ఒక్కటి వచ్చిన క్షణం –

```
[10, 9, 1, 100, 25].sort() → [1, 10, 100, 25, 9]
```

"మీ CODE ని అనుమానించే ముందు మీ DATA ని అనుమానించండి"

Deep 03 lo ఒక-అంకె జతలు hash key bug ని దాచాయి · **Deep 05** lo దట్టమైన graphs $n-1$ mutation ని దాచాయి · **Deep 06** lo $n \leq 12$ arrays పని-పరిమితిని నిరుపయోగం చేశాయి · **Deep 08** lo 1 లేని నాణెం వ్యవస్థలే లేవు.

ఇది అయిదోసారి. మరియు ప్రతిసారీ ఆకారం ఒకటే: సరైన code కి మరియు తప్పు code కి ఒకే జవాబు ఇచ్చే data.

కాబట్టి ఒక పరీక్ష రాసిన తర్వాత అడగండి: "ఈ data ఏ తప్పుని *చూపించలేదు*?"

5. Step – $a > b$ ఎందుకు ఏమీ చేయదు

సరే, comparator ఇద్దాం:

```
arr.sort((a, b) => a > b); // ← చాలామంది రాసేది
```

boolean comparator `arr.sort((a,b) => a > b)`

n	2,000 arrays lo తప్పుగా sort అయినవి	%
4	1,926	96.3%
8	2,000	100.0%
32	2,000	100.0%
1000	2,000	100.0%

5,000 ARRAYS LO 5,000 సార్లు OUTPUT = INPUT – అంటే 100%

$a > b$ ఇచ్చేది `true` లేదా `false`. Sort అది సంఖ్యగా మార్చుతుంది: **1 లేదా 0.**

ఋణాత్మక సంఖ్య ఎప్పుడూ రాదు. కాబట్టి sort కి " a, b కంటే చిన్నది" అని తెలిసే మార్గమే లేదు – అది ప్రతి జతన "సమానం లేదా పెద్దది" అని చూస్తుంది.

ఫలితం: **array అస్సలు కదలదు.** Crash లేదు, హెచ్చరిక లేదు – మీ data యథాతథంగా తిరిగి వస్తుంది, మరియు మీరు దాన్ని "sorted" అనుకుంటారు.

సరైనది: $(a, b) => a - b$ – ఇది ముగ్గురినీ ఇస్తుంది: ఋణాత్మకం, సున్నా, ధనాత్మకం.

Part 3 – రెండో విరుపు: స్థిరత్వం

6. Step – రెండు keys ప్రకారం sort

ఒక leaderboard: **score** ప్రకారం (ఎక్కువ ముందు), సమాన scores lo పేరు ప్రకారం.

సహజమైన పద్ధతి – **రెండు దశలు**: ముందు పేరు ప్రకారం, ఆపై score ప్రకారం. Sort స్థిరమైతే సమాన scores lo పేర్ల క్రమం నిలిచిపోతుంది.

```
const byName = [...players].sort((a, b) => a.name < b.name ? -1 : a.name > b.name ? 1 : 0);  
return byName.sort((a, b) => a.score < b.score ? 1 : -1); // ఎక్కువ score ముందు
```

`Array.sort` ES2019 నుంచి స్థిరమైనది. కాబట్టి ఇది పనిచేయాలి.

7. రెండో విరుపు – 21,000 runs, 0 తప్పులు, 100% నష్టం

Leaderboard – ముందు పేరు ప్రకారం, ఆపై score ప్రకారం

అశించినది (a.score - b.score) | వచ్చినది (? 1 : -1)

90	Anil	<	90	Eshwar
90	Charan		90	Charan
90	Eshwar	<	90	Anil
85	Bhavani	<	85	Geetha
85	Deepa		85	Deepa
85	Geetha	<	85	Bhavani
70	Farhan	<	70	Harsha
70	Harsha	<	70	Farhan

8 వరుసల్లో 6 తప్పు స్థానంలో. కానీ scores మాత్రం సరిగ్గానే పేరుకున్నాయి.

comparator (a,b) => a.score > b.score ? 1 : -1 - 0 ఎప్పుడూ ఇవ్వదు

n | ప్రత్యేక scores | 3,000 runs: sort తప్పు | స్థిరత్వం పోయినవి

10		10		0		2,999
10		3		0		3,000
50		50		0		3,000
50		5		0		3,000
200		200		0		3,000
200		10		0		3,000
1000		20		0		3,000

అదే data, comparator (a,b) => a.score - b.score :

200	10	0	0
1000	20	0	0

21,000 RUNS • SORT తప్పుంది నున్నా సార్లు • స్థిరత్వం పోయింది 100%

మీరు రాసే సహజమైన పరీక్ష – “ఫలితం sort అయిందా?” – ఈ bug ని **ఎప్పటికీ** పట్టుకోదు. 21,000 ప్రయత్నాల్లో ఒక్కసారి కాదు.

కారణం: ? 1 : -1 lo **0 ఎప్పుడూ రాదు**. సమాన scores ఉన్న రెండు మూలకాలకి comparator “రెండోది ముందు” అని చెబుతుంది – **ఏ క్రమంలో అడిగినా**. అంటే $\text{cmp}(a,b) = -1$ మరియు $\text{cmp}(b,a) = -1$.

అది §1 యొక్క **వ్యతిరేకత** నియమాన్ని ఉల్లంఘిస్తుంది. Spec ప్రకారం ఫలితం undefined. V8 lo array పాడవదు, మూలకాలు పోవు – **కేవలం సమాన మూలకాల క్రమం అర్థరహితం అవుతుంది**.

మరియు అదే మీరు రెండు-దశల sort lo ఆధారపడుతున్న ఒక్క విషయం.

స్థిరమైన SORT + అస్థిరమైన COMPARATOR = అస్థిరమైన ఫలితం

`Array.sort` స్థిరమైనది అనేది **నిజం**. కానీ “స్థిరం” అంటే “comparator సమానం అన్న మూలకాల క్రమం నిలుపుతాను” అని అర్థం.

మీ comparator **ఏదీ సమానం అనకపోతే** – నిలపడానికి ఏమీ లేదు.

స్థిరత్వం sort యొక్క లక్షణం కాదు. అది sort మరియు comparator కలిసి ఇచ్చే లక్షణం.

8. Step – byKeys

రెండు-దశల sort పని చేస్తుంది, కానీ ప్రతిసారి ఆ ఒప్పందం గుర్తు పెట్టుకోవాలి. ఒక్క comparator lo రాయడం స్పష్టం:

```
function byKeys(keys) {                                     // keys: [{ get, desc }]
  return (a, b) => {
    for (const k of keys) {
      const x = k.get(a), y = k.get(b);
      if (x < y) return k.desc ? 1 : -1;
      if (x > y) return k.desc ? -1 : 1;
    }
    return 0;                                               // ← ఇది *తప్పక* 0. §7 యొక్క bug సరిగ్గా ఇక్కడే పుడుతుంది.
  };
}
```

`byKeys([ {get: p => p.score, desc: true}, {get: p => p.name} ])` – ఒకే pass, స్థిరత్వం మీద ఆధారపడదు, మరియు చదివితే **ఏ క్రమం కావాలో స్పష్టంగా** తెలుస్తుంది.

§12 lo ఒక నియమం ఈ రెండు దారులూ **ఒకే ఫలితం** ఇస్తాయని ప్రతి run lo తనిఖీ చేస్తుంది.

Part 4 – మూడో విరుపు: partition

9. Step – quicksort మరియు pivot

పాఠ్యపుస్తక quicksort – చివరి మూలకమే pivot:

quicksort (చివరి మూలకం pivot) • పోలికల సంఖ్య

n	యాదృచ్ఛిక input	ఇప్పటికే sorted input	రెట్లు
100	584	4,950	8.5×
500	5,138	1,24,750	24.3×
2000	24,691	19,99,000	81.0×
5000	68,152	1,24,97,500	183.4×

quicksort • ఇప్పటికే sorted input • default stack

n	ఫలితం
1000	సరే • recursion లోతు 999
2000	సరే • recursion లోతు 1999
4000	RangeError: Maximum call stack size exceeded

ఇప్పటికే SORTED ARRAY → RECURSION లోతు N-1 → N = 4,000 దగ్గర CRASH

Sorted input లో చివరి మూలకం ఎప్పుడూ **అతి పెద్దది**. కాబట్టి ప్రతి partition $n-1$ మరియు 0 గా విడుతుంది – ఇది quicksort కాదు, **selection sort**.

మరియు అది నెమ్మదవ్వడం కంటే **ముందే** crash అవుతుంది: $n = 4,000$ దగ్గర stack నిండిపోతుంది. **Deep 04** లో ఇదే RangeError, అదే కారణం.

పరిష్కారం రెండు ముక్కలు: (1) median-of-three pivot – lo, mid, hi ల మధ్యస్థాన్ని తీసుకోవడం • **(2)** చిన్న భాగాన్ని మాత్రమే recursion, పెద్దదాన్ని loop లో – దీనితో లోతు **ఎప్పుడూ $\leq \log_2 n$** .

రెండూ కలిపితే sorted input **$0.96 \times n \log n$** కి వస్తుంది. సమస్య పరిష్కారం అయినట్టే.

10. మూడో విరుపు – median-of-three నకళ్ళని చూడడు

§12 కోసం fuzz రాసినప్పుడు ఒక **పని-పరిమితి** పెట్టాను – “ $quickSort$ పోలికలు $\leq 3 \cdot n \cdot \lceil \log_2(n+1) \rceil + 20$ ”. ఆపై నా స్వంత code మీద నడిపాను:

$n = 2,000$ · పోలికల సంఖ్య ($n \cdot \log_2 n = 21,932$)

input ఆకారం	Lomuto (2-భాగాలు)	$\div n \log n$	మూడు-భాగాలు	$\div n \log n$
యాదృచ్ఛికం	23,219	1.06×	24,801	1.13×
ఇప్పటికే sorted	21,033	0.96×	37,976	1.73×
నకళ్ళు (0-3)	5,11,747	23.33×	4,996	0.23×
అన్నీ ఒకే విలువ	20,04,997	91.42×	2,003	0.09×

MEDIAN-OF-THREE SORTED INPUT ని సరిచేసింది. నకళ్ళని అది చూడనే లేదు.

Lomuto partition lo షరతు `cmp(arr[j], p) < 0`. సమానమైన ప్రతి మూలకం కుడి వైపుకి వెళ్తుంది.

కాబట్టి అన్నీ ఒకే విలువ ఉన్న array lo pivot ఏదైనా సరే – ఎడమ భాగం ఎప్పుడూ ఖాళీ. Median-of-three మంచి pivot ఎంచుకుంటుంది, మరియు అది ఏమీ మార్చదు.

91.42× – మరియు ఇది నా code, నేను "సరిచేశాను" అనుకున్న తర్వాత.

అన్నీ ఒకే విలువ · n పెరిగితే

n	Lomuto పోలికలు	మూడు-భాగాలు	రెట్లు
250	31,872	253	126×
500	1,26,247	503	251×
1000	5,02,497	1,003	501×
2000	20,04,997	2,003	1001×
4000	80,09,997	4,003	2001×

రెట్లు ప్రతిసారీ రెట్టింపు అవుతున్నాయి – అదే $O(N^2)$ యొక్క సంతకం

$126 \rightarrow 251 \rightarrow 501 \rightarrow 1001 \rightarrow 2001$. n రెట్టింపైతే నిష్పత్తి రెట్టింపు.

అంటే ఒకటి n తో పెరుగుతోంది, ఇంకొకటి n^2 తో. ఒక్క సంఖ్య ఇది చెప్పుదు – వరుస చెబుతుంది.

పరిష్కారం – మూడు-భాగాల partition (Dutch national flag): `< p`, `= p`, `> p`. సమానమైనవన్నీ మధ్యలో ఉండిపోతాయి, recursion కి వెళ్ళవు. అన్నీ ఒకటే అయితే ఒక్క pass చాలు – 4,003 పోలికలు, అంటే n తో సమానం.

ఖరీదు: మూడు-భాగాలు SORTED INPUT మీద 1.73×, రెండు-భాగాలు 0.96×

దాచడం లేదు – మూడు-భాగాల partition ఉచితం కాదు. Sorted input మీద ఇది దాదాపు రెండింతలు పోలికలు చేస్తుంది, యాదృచ్ఛిక input మీద 1.06× కి బదులు 1.13×.

మీరు కొంటున్నది: నకళ్ళ దగ్గర 91× నుంచి 0.09× కి.

మీ data lo నకళ్ళు లేవని *నిజంగా* తెలిస్తే రెండు-భాగాలే మంచిది. తెలియకపోతే – 1.73× ఒక చౌక బీమా.

Part 5 – నిరూపణ మరియు ప్రదర్శన

11. మొత్తం code

```
// మూడు భాగాలు: [lo,lt) < p · [lt,i) = p · (gt,hi] > p → నకళ్ళకి రక్ష
let lt = lo, i = lo, gt = hi;
while (i <= gt) {
  const d = cmp(arr[i], p);
  if (d < 0) { [arr[lt], arr[i]] = [arr[i], arr[lt]]; lt++; i++; }
  else if (d > 0) { [arr[i], arr[gt]] = [arr[gt], arr[i]]; gt--; }
  else i++; // సమానం - అది ఇక్కడే ఉండిపోతుంది, recursion కి వెళ్ళదు
}
// చిన్న భాగాన్ని recursion, పెద్దదాన్ని loop → recursion లోతు ≤ log₂n
if (lt - lo < hi - gt) { go(lo, lt - 1); lo = gt + 1; }
else { go(gt + 1, hi); hi = lt - 1; }
```

```
// స్థిరమైన merge sort - '<=' వల్లే స్థిరం
if (cmp(L[i], R[j]) <= 0) out.push(L[i++]); // సమానమైతే *ఎడమది* ముందు
else out.push(R[j++]);
```

ఆ `<=` ని `<` గా మార్చితే ఫలితం ఇంకా **sort అయే ఉంటుంది** - స్థిరత్వం మాత్రమే పోతుంది. §12 lo అది **120/120** విఫలం, ఎందుకంటే ఒక నియమం సరిగ్గా అదే అడుగుతుంది.

12. 24,000 cases • mutation testing

పదకొండు నియమాలు: **mergeSort = built-in, quickSort = built-in, input మారలేదు, mergeSort పోలికలు $\leq n \lceil \log_2 n \rceil$, quickSort పోలికలు $\leq 3n \lceil \log_2(n+1) \rceil + 20$, countingSort = built-in, ప్రతి విలువ లెక్క అలాగే, mergeSort స్థిరం, objects sort అయ్యాయి, ఒక్క మూలకమూ పోలేదు, byKeys = రెండు-దశల స్థిర sort, byKeys సమానమైతే 0.**

```
300 runs · 12,000 arrays · 6,000 counting sorts · 6,000 object sorts
నియమ ఉల్లంఘనలు: 0
```

ఏ దారులు నడిచాయి:

arrays	12,000
objs	6,000
counting	6,000
dupHeavy	4,215
presorted	2,430
big (n ≥ 100)	6,174
multiKey	6,000

==== mutation testing ====

మార్పు లేని code → 0/300 విఫలం
merge lo `cmp(L[i],R[j]) <= 0` ని `< 0` గా → 120/120 విఫలం
ఉదా: mergeSort స్థిరం కాదు: idx 28 ముందు 27
mergeSort lo `mid = arr.length >> 1` ని `mid = 1` గా → 120/120 విఫలం
ఉదా: mergeSort 633 పోలికలు > పరిమితి 270 (n=45)
quickSort lo median-of-three తీసేస్తే (చివరి మూలకమే pivot) → 120/120 విఫలం
ఉదా: quickSort 38,216 పోలికలు > పరిమితి 7,391 (n=273)
quickSort lo మూడు భాగాలకి బదులు రెండు భాగాలు → 120/120 విఫలం
ఉదా: quickSort 24,75,00,000 పోలికలు > పరిమితి 3,908 (n=162)
quickSort lo చిన్న భాగాన్ని recursion చేసే ఎంపిక తీసేస్తే → 0/120 విఫలం
countingSort lo `cnt[v]++` ని `cnt[v] = 1` గా → 120/120 విఫలం
ఉదా: countingSort ≠ built-in · [9,9,13,2,9,11,13,6] → [0,1,2,3,4,5,6,7]
countingSort lo `v <= maxV` ని `v < maxV` గా → 120/120 విఫలం
ఉదా: countingSort lo 15 : 6 → undefined
byKeys lo `return 0` ని `return 1` గా (§7 యొక్క bug) → 120/120 విఫలం
ఉదా: byKeys ≠ రెండు-దశల స్థిర sort · n=31
byKeys lo keys క్రమాన్ని తిరగేస్తే → 120/120 విఫలం

రెండు-భాగాల MUTATION ముగియనే ముగియదు

24.75 కోట్ల పోలికలు అనే సంఖ్య నిజమైనదే, కానీ అది **సహజంగా ఆగిన** run కాదు – నేను ఒక loop guard పెట్టాకే కొలవగలిగాను.

కారణం: సమానమైనవన్నీ కుడి వైపుకి వెళ్తే, అన్నీ ఒకే విలువ ఉన్న array lo lt = lo మరియు gt = lo-1 అవుతాయి. అప్పుడు lo ముందుకు కదలదు, మరియు while (lo < hi) ఎప్పటికీ ఆగదు.

ఒక "కేవలం నెమ్మది" అనుకున్న మార్పు నిజానికి ఒక అనంత loop. §10 యొక్క 91.42× ని Lomuto (arr[i] తో swap చేసేది) ఇస్తుంది; నా మూడు-భాగాల code నుంచి మూడో శాఖ తీసేస్తే అది నెమ్మది కాదు – అది hang.

బతికిన MUTATION – "చిన్న భాగాన్ని RECURSION" తీసేసినా 0/120

ఆ పంక్తి recursion లోతుని కాపాడుతుంది. నా నియమాలు పోలికలు లెక్కపెడతాయి, లోతు కాదు – మరియు 400 మూలకాల array stack ని నింపదు.

కాబట్టి కొలిచాను – $n = 1,00,000$ దగ్గర గరిష్ట లోతు:

input ఆకారం	చిన్న భాగం recursion	ఎప్పుడూ ఎడమ భాగం
యాదృచ్ఛికం	11	21
sorted	14	16
reverse	14	27
నకళ్ళు 0-3	2	2
అన్న ఒకటే	1	1

తేడా ఉంది (11 vs 21), కానీ crash కి వేలల్లో లోతు కావాలి, మరియు నేను ప్రయత్నించిన ఏ ఆకారమూ దాన్ని దగ్గరకైనా తీసుకెళ్ళలేదు.

తీర్పు: median-of-three మరియు మూడు-భాగాల partition *కలిసి* ఆ పంక్తి కాపాడే సందర్భాన్ని ఇప్పటికే నివారిస్తున్నాయి. అది రక్షణ కోసం ఉంది – నేను నిర్మించగలిగిన ఏ input మీదా అది భారం మోయడం లేదు. దాచడం కాదు; నేను దాన్ని విరగొట్టే input తయారు చేయలేకపోయాను.

13. $n \log n$ ని దాటడం

పోలికల మీద ఆధారపడే ఏ sort అయినా $\Omega(n \log n)$ – ఇది నిరూపితం. కానీ ఆ నిరూపణ "పోలికలు మాత్రమే వాడితే" అనే షరతు మీద ఆధారపడుతుంది.

విలువలు చిన్న పూర్ణాంక పరిధిలో ఉంటే – పోలికలే అక్కర్లేదు:

పనితీరు · $n = 20,00,000$ · విలువలు 0–999

```
.sort((a,b)=>a-b) : 251 ms
counting sort      : 15 ms → 16.7× వేగం
ఒకే ఫలితమా? true
```

Ω(N LOG N) ని "దాటడం" లేదు – దాని షరతు నుంచి బయటపడటం

Counting sort ఏ రెండు మూలకాలనీ పోల్చదు. అది **లెక్కపెడుతుంది**: విలువ 7 ఎన్నిసార్లు వచ్చిందో ఒక array lo పెట్టి, ఆపై 0 నుంచి 999 వరకు నడుస్తుంది.

ఖరీదు: $\text{maxV} + 1$ **పరిమాణం ఉన్న ఒక array**. విలువలు 0-999 అయితే అది 1,000 గళ్ళు – చౌక. విలువలు 0-10⁹ అయితే అది **4 GB**, మరియు ఇది పనికిరాదు.

కాబట్టి §2 యొక్క ప్రశ్న: **"keys ఒక చిన్న పూర్ణాంక పరిధిలో ఉన్నాయా?"** – అవును అంటే 16.7×, కాదంటే ఈ ఆలోచన వదిలేయండి.

14. Follow-ups – external sort, radix, TimSort

"Data RAM lo పట్టకపోతే?"

External merge sort: data ని RAM lo పట్టే ముక్కలుగా చదివి, ఒక్కో ముక్కనీ sort చేసి డిస్క్కి రాసి, ఆపై అన్ని ముక్కలనీ ఒక k-way merge (Deep 06 యొక్క heap) తో కలపడం. **నేను దీన్ని కొలవలేదు** – HLD Deep 15 lo డిస్క్ vs RAM సంఖ్యలు ఉన్నాయి.

"Radix sort counting sort కంటే మంచిదా?"

Radix sort counting sort ని **ఒక్కో అంకె మీద** పదే పదే నడుపుతుంది, కాబట్టి maxV పెద్దదైనా memory పేలదు. ఖరీదు: d passes, d = అంకెల సంఖ్య. **ప్రతి pass స్థిరమైనదై తీరాలి** – లేకపోతే మొత్తం విరుగుతుంది, మరియు ఇది §7 యొక్క పాఠమే మళ్ళీ. **నేను radix sort ని కొలవలేదు.**

"V8 ఏ algorithm వాడుతుంది?"

TimSort – merge sort మరియు insertion sort కలయిక. ఇది ఇప్పటికే sorted ముక్కలని ("runs") గుర్తించి వాడుకుంటుంది, కాబట్టి దాదాపు sorted data మీద **O(n)**. మరియు ఇది **స్థిరమైనది** – §7 ఆధారపడేది సరిగ్గా దీని మీదే.

"అన్నీ ఒకటే అనే array నిజ జీవితంలో వస్తుందా?"

§10 lo అది ఒక తీవ్రమైన సందర్భం. కానీ **నకళ్ళు (0-3)** వరుస – 23.33× – అది రోజువారీ: "status ప్రకారం sort చెయ్యి" (pending / done / failed), "నక్షత్రాల ప్రకారం" (1-5), "విభాగం ప్రకారం". **కొన్ని ప్రత్యేక విలువలు, లక్షల rows** – అదే అత్యంత సాధారణ ఆకారం.

"ఈ doc lo నేను ఏమి కొలవలేదు"

మూడు: **external sort, radix sort**, మరియు **V8 యొక్క TimSort** – నేను నా స్వంత merge sort మరియు quicksort ని మాత్రమే కొలిచాను; §13 యొక్క 251 ms మాత్రం నిజమైన V8 sort.

15. ఏమి నేర్చుకున్నాం

1. మీరు రాసేది sort కాదు – comparator. మూడు విరుపుల్లో రెండు అందులోనే.
2. `.sort()` comparator లేకుండా అన్నిటినీ string గా చూస్తుంది. ఒక-అంకె data మీద 0.0% తప్పు, నిజమైన data మీద 63.3%.
3. `(a, b) => a > b` array ని కదపదు. 5,000 arrays lo 5,000 సార్లు output = input. `a - b` రాయండి.
4. `? 1 : -1` అనే comparator 21,000 runs lo ఒక్క array నీ తప్పుగా sort చేయలేదు – మరియు స్థిరత్వాన్ని 100% నాశనం చేసింది.
5. స్థిరత్వం sort యొక్క లక్షణం కాదు. అది sort మరియు comparator కలిసి ఇచ్చేది. `return 0` లేకపోతే నిలపడానికి ఏమీ లేదు.
6. Sorted input quicksort ని నెమ్మది చేయదు – దాన్ని crash చేస్తుంది. $n = 4,000$ దగ్గర `RangeError`, recursion లోతు $n-1$.
7. Median-of-three sorted input ని సరిచేస్తుంది, నకళ్ళని చూడనే చూడదు. అన్నీ ఒకే విలువ $\rightarrow 91.42\times$, మరియు n రెట్టింపైతే నిప్పత్తి రెట్టింపు.
8. మూడు-భాగాల partition ఉచితం కాదు – sorted input మీద $0.96\times$ నుంచి $1.73\times$. అది నకళ్ళ దగ్గర $91\times$ కి కొన్న బీమా.
9. $\Omega(n \log n)$ ని దాటలేరు, కానీ దాని షరతు నుంచి బయటపడొచ్చు. Counting sort $16.7\times$, ఖరీదు `maxV+1` గళ్ళ array.
10. ఒక-అంకె data ఏ తప్పున చూపించదు. Deep 03, 05, 06, 08 – ఇది అయిదోసారి.

ఈ DOC నుంచి ఒక్క వాక్యం గుర్తుపెట్టుకోవాలంటే

తప్పు comparator sort ని విఫలం చేయదు – అది sort ని అర్థరహితం చేస్తుంది, మరియు ఫలితం sorted గానే కనిపిస్తుంది.

§4 lo array కదలనే లేదు. §7 lo scores సరిగ్గా పేరుకున్నాయి, పేర్లు అర్థరహితం. §10 lo ఫలితం ఖచ్చితంగా సరైనది – $2,001\times$ నెమ్మదిగా.

మూడూ "sort పనిచేసింది" అని చెబుతాయి. కాబట్టి "sort అయిందా?" అని అడగడం సరిపోదు. అడగండి: సమానమైనవి ఏమయ్యాయి? మరియు ఎన్ని పోలికలు తీసుకుంది?